SHERLOCK

Security Review For Centrifuge



Collaborative Audit Prepared For: **Centrifuge**
Lead Security Expert(s): **0x52**
Date Audited: **September 26 - September 28, 2026**

Introduction

v3.3.0 hardens the pool-management and cross-chain surface introduced across v3.1–v3.2 and makes the protocol's core primitives more flexible. The main additions are a per-pool manifest policy layer that constrains manager actions, fully sovereign per-pool adapter sets, and per-chain risk controls that bound the damage a single compromised chain can do. This release also ships pluggable custom share tokens, a redesigned balance sheet now folded into Spoke, and flexible accounting.

Scope

Repository: centrifuge/protocol-internal

Audited Commit: 544456401db49cf9c9999293578f1abedb4c02ef

Final Commit: 051718ed55e3db59ed713596441e1dc837b8d461

Files:

- src/adapters/AxelarAdapter.sol
- src/adapters/ChainlinkAdapter.sol
- src/adapters/HyperlaneAdapter.sol
- src/adapters/LayerZeroAdapter.sol
- src/adapters/StandbyAdapter.sol
- src/admin/GasService.sol
- src/admin/OpsGuardian.sol
- src/admin/ProtocolGuardian.sol
- src/core/hub/Holdings.sol
- src/core/hub/HubHandler.sol
- src/core/hub/HubRegistry.sol
- src/core/hub/Hub.sol
- src/core/hub/ShareClassManager.sol
- src/core/messaging/Gateway.sol
- src/core/messaging/MessageProcessor.sol
- src/core/messaging/MultiAdapter.sol
- src/core/spoke/Escrow.sol
- src/core/spoke/factories/EscrowFactory.sol
- src/policies/hub/StdHubPolicy.sol

- src/token/hooks/BaseTransferHook.sol

Final Commit Hash

051718ed55e3db59ed713596441e1dc837b8d461

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	2

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: Future timestamps can permanently DoS normal price updates [RESOLVED]

Source: https://github.com/sherlock-audit2/2026-09-centrifuge-update-sep-26th-2026/issues/21

Summary

`SpokeRegistry.updatePricePoolPerShare` and `updatePricePoolPerAsset` accept future `computedAt` values and reject subsequent timestamps below the stored value. A compromised Gateway manager on a Spoke can inject `computedAt = type(uint64).max`, permanently denying service to normal price updates for that entry over the deployment's practical lifetime.

Revoking the manager does not end the DoS: legitimate notifications continue reverting with `CannotSetOlderPrice`. Even Root must satisfy the same comparison and has no timestamp reset method. Root can manually correct the price using the poisoned timestamp, but restoring normal timestamped updates requires replacing the registry and rewiring its consumers.

Code Snippet

SpokeRegistry.sol#L353-L376

SpokeHandler.sol#L179-L188

Gateway.sol#L109-L137

ShareClassManager.sol#L65-L75

Tool Used

Manual Review

Recommendation

Reject timestamps beyond a documented clock-skew allowance and add a Root-authorized timestamp correction method. Contain the compromised message source and handle outstanding messages before correction so they cannot restore the bad timestamp.

Discussion

hieronx

Fixed in https://github.com/centrifuge/protocol-internal/pull/425

Issue L-2: SimplePriceManager cannot recover from issuance-delta overflow [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit2/2026-09-centrifuge-update-sep-26th-2026/issues/22

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`SimplePriceManager._calculateIssuanceDelta` evaluates `oldIssuance + transferredIn` and `newIssuance + transferredOut` as `uint128` sums before subtracting them. Once the stored old issuance and incoming-transfer counter sum to more than `type(uint128).max`, every update for that pool/network reverts before SPM can replace its cached issuance or clear its counters, even when the correct net adjustment is zero.

A privileged message injector can create this state if the Hub bridging hook is absent or permits the amount. Starting from positive synchronized issuance `S` and zero transfer counters on a remote network, a forged `InitiateTransferShares` with that network as both source and destination and amount `type(uint128).max` leaves SCM's net issuance unchanged while filling both SPM counters. HubHandler accepts equal source and destination, and forwarding to the remote network does not require a successful destination mint before the Hub writes commit. The next SPM update attempts `S + type(uint128).max` and reverts.

Revoking the injector, replacing adapters, or correcting SCM's current issuance cannot repair the stored sum in SPM. Root has no reset method and must bypass or replace SPM, reconciling the replacement's metrics and permissions. A failing snapshot also rolls back its nonce increment and blocks that ordered accounting stream until recovery; restrictive Hub bridge limits can prevent the example above.

Code Snippet

SimplePriceManager.sol#L34-L78

SimplePriceManager.sol#L92-L107

HubHandler.sol#L117-L155

ShareClassManager.sol#L105-L115

MessageDispatcher.sol#L465-L491

Holdings.sol#L100-L112

Tool Used

Manual Review

Recommendation

Compute the adjusted sums and their difference in `uint256`, checking the final delta before narrowing. Add a Root-authorized reconciliation method that keeps cached transfer counters, per-network issuance and aggregate metrics consistent. Reject equal source and destination in HubHandler before recording a transfer.

Discussion

hieronx

Ack, will be fixed in a future release.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.